Testing a Python API

> Part of the Python API Backend curriculum. This document covers `pytest` fundamentals, FastAPI's `TestClient`, fixture-based test data (including fixtures produced by running the real upstream pipeline), mocking, and the discipline of proving a test actually catches the bug it claims to.

1. Why this layer is well-suited to automated tests

Unlike exploratory notebook code (Data Science / ML, EDA doc, §8), an API has a **stable, callable contract**: a fixed set of routes, each with a defined request shape and response shape. That's exactly the kind of thing automated tests are good at protecting — you can call an endpoint and assert on its status code and response shape without needing to re-derive any analysis logic.

2. pytest fundamentals

```python
def add(a: int, b: int) -> int:
    return a + b

def test_add():
    assert add(2, 3) == 5
```

A `pytest` test is just a function whose name starts with `test_`, containing plain `assert` statements. Run the whole suite with `pytest`; run a single file with `pytest path/to/test_file.py`; run a single test with `pytest path/to/test_file.py::test_name`.

2.1 Fixtures

A **fixture** is reusable setup/teardown logic, declared once and requested by name in any test that needs it:

```python
import pytest

@pytest.fixture
def sample_data():
    return {"a": 1, "b": 2}

def test_uses_fixture(sample_data):
    assert sample_data["a"] == 1
```

Fixtures can depend on other fixtures (just by naming them as parameters), and can run teardown code after the test via `yield` instead of `return`:

```python
@pytest.fixture
def tmp_resource():
    resource = acquire_resource()
    yield resource            # the test runs here, with `resource` as its value
    release_resource(resource)  # runs after the test, whether it passed or failed
```

2.2 Parametrization

Run the same test logic against several inputs without copy-pasting the test function:

```python
@pytest.mark.parametrize(
    ("a", "b", "expected"),
    [(1, 1, 2), (2, 3, 5), (-1, 1, 0)],
)
def test_add_parametrized(a, b, expected):
    assert add(a, b) == expected
```

This produces three separate, independently-reported test cases from one function definition — useful whenever you have several concrete examples of the same underlying behavior to check (e.g., several distinct malformed-input shapes that should all produce the same `422` response).

3. Testing FastAPI endpoints with `TestClient`

FastAPI exposes a `TestClient` (built on `httpx`) that lets you call your app directly in a test, without a real running server:

```python
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_get_item_happy_path():
    response = client.get("/items/abc123")
    assert response.status_code == 200
    body = response.json()
    assert body["id"] == "abc123"

def test_get_item_unknown_id_is_404():
    response = client.get("/items/does-not-exist")
    assert response.status_code == 404
```

4. Test data: never test against real production data

Since the API just reads whatever files/database the data layer produced, tests need their own small, controlled dataset rather than depending on real (often large, sometimes gitignored, always slow-to-regenerate) production output.

4.1 The fixture-CSV-and-monkeypatch pattern

1. Write small, hand-built fixture data (a few rows, deliberately including edge cases — a missing value, an out-of-range entry, a value right at a boundary condition) to a temporary directory for each test.
2. **Monkeypatch** the one configuration point the data-loading code reads from (e.g., a `DATA_DIR` constant) to point at that temporary directory, rather than mocking the loader functions themselves — this way, the *real* loading/filtering code still runs during the test, so you're verifying actual logic, not a stand-in for it.
3. Clear any `@lru_cache`'d function's cache (`.cache_clear()`) before each test, so one test's fixture data can't leak into the next test via a stale cached result.

```
import pytest

@pytest.fixture
def data_dir(tmp_path, monkeypatch):
    monkeypatch.setattr(app.data_loaders, "DATA_DIR", str(tmp_path))
    app.data_loaders.load_dataset.cache_clear()
    yield tmp_path
    app.data_loaders.load_dataset.cache_clear()

@pytest.fixture
def full_data(data_dir):
    pd.DataFrame({"id": ["abc123"], "value": [42.0]}).to_csv(data_dir / "dataset.csv", index=False)
    return data_dir

@pytest.fixture
def client(full_data):
    return TestClient(app)
```

`tmp_path` and `monkeypatch` are both **built-in pytest fixtures** — `tmp_path` gives you a fresh, automatically-cleaned-up temporary directory per test; `monkeypatch` gives you a safe way to modify module attributes, environment variables, or dictionary entries for the duration of a single test, automatically reverting the change afterward regardless of whether the test passes or fails.

4.2 Mocking

Mocking replaces a real dependency with a controllable stand-in, so a test can force a specific scenario (a slow call, a failure, a specific return value) without needing to actually trigger it:

```
from unittest.mock import patch

def test_handles_external_service_failure():
    with patch("app.services.call_external_api", side_effect=TimeoutError):
        response = client.get("/items/abc123")
        assert response.status_code == 503
```

Prefer the fixture-and-monkeypatch approach (§4.1) over mocking your *own* data-loading logic directly — mocking the loader itself would skip over exactly the filtering/parsing code you actually want tested. Reach for mocking when the dependency is a genuinely external thing (a network call to another service, a slow or non-deterministic operation) that you can't or shouldn't actually invoke in a fast, deterministic test.

4.3 Build the fixtures by running the real producer

Hand-written fixture files drift. If the API reads files a pipeline writes, the strongest fixture is **the pipeline's own output**: have the test suite run the real pipeline stages on small, stubbed inputs (a few years of fake data per source) into a temp directory, and point the API's data-directory setting at it. Now a schema change in the pipeline breaks the *API's* tests — in CI, immediately — rather than breaking production on the next scheduled refresh. It costs a slightly slower suite and a producer that can run on stubbed inputs, both worth having anyway. Reserve hand-written fixtures for the cases the real producer cannot emit (a corrupt file, a non-finite number, an unsupported `schema_version`) and write one test per entry in the fail-closed table from Design, §2.3.

5. Testing error paths, not just happy paths

For every required data dependency, write a test that simply doesn't write that fixture file, and confirms the endpoint responds with the expected `503` and a useful message — this is just as important as testing the successful case, since it's the path a real deployment will hit whenever an upstream step hasn't run yet:

```
def test_endpoint_returns_503_when_data_missing(data_dir):
    # deliberately don't write the fixture file
    response = TestClient(app).get("/items")
    assert response.status_code == 503
    assert "not found" in response.json()["detail"]
```

6. Prove your tests actually catch the bug they claim to

A test that passes today isn't automatically protecting against the regression it's meant to catch. When you write a test for a specific bug class (e.g., the route-ordering issue in Fundamentals, §4.1, or a path-boundary issue in middleware), **deliberately reintroduce the bug locally and confirm the test fails** before trusting it, then fix the code back:

```
1. Write the test.
2. Confirm it passes against the current, correct code.
3. Temporarily reintroduce the specific bug (e.g., swap route declaration order).
4. Re-run the test — it MUST fail now, with a clear message.
5. Revert the deliberate bug.
6. Re-run the test — it should pass again.
```

This also surfaces a subtle trap: two different implementations (one correct, one subtly broken) can sometimes produce the *same* final outcome for a given test input, just for different reasons — e.g., a malformed request might 404 either way: correctly rejected, or incorrectly mangled into nonsense that also happens to 404. If a test doesn't actually change outcome when you break the code, it isn't testing what you think it's testing; you may need a more targeted input that only the correct implementation handles properly (for example, an input that a *naive* buggy implementation would incorrectly accept and mangle into something that coincidentally still resolves, versus a correct implementation that properly leaves it alone).

7. Beyond unit/integration tests: load testing

For an API expected to handle meaningful concurrent traffic, functional correctness tests (everything above) don't tell you how the service behaves *under load*. Tools like Locust let you script realistic request patterns and ramp up concurrent simulated users to observe response times and failure rates as load increases — worth reaching for once an API moves from "internal tool with a handful of users" toward anything with real traffic expectations.

8. Core libraries summary

Library	Used for
pytest	the test runner, fixtures, parametrization
FastAPI's `TestClient` (via `httpx`)	calling the app directly in tests, without a running server
`unittest.mock` (standard library)	mocking external dependencies
Locust (or similar)	load/performance testing

See also

- Python API Backend index
- API Design & Best Practices — the error-handling and data-loading patterns these tests exercise
- Deployment — running the tested application in production